COMP 3004 Fall 2026

DragonEndPOS Deliverable 1

Due: Thursday, Nov 12, 2026, at 23h59

Collaboration: This deliverable must be completed by registered teams

Overview

Your team will design and build the first working version (described in Part 2) of the **Dragon's End Café Point of Sales System (DragonEndPOS).**

This deliverable focuses on defining a **system decomposition diagram** and producing a **functional in-memory vertical prototype** that demonstrates the customer viewing the menu, placing an online order, and viewing their online dashboard, and the kitchen staff viewing the order queue and completing orders.

This deliverable builds upon the *DragonEndPOS Project Overview document*. Database or file persistence is outside of the scope for D1; **your system operates entirely in memory.**

Part 1 – System Decomposition Diagram (20 marks)

Submission must include the following in one PDF document:

1. A UML system decomposition diagram, embedded in the PDF **(not as a link)**, with a descriptive figure title that names your architectural style (e.g., “Figure 1: DragonEndPOS System Decomposition Based on a Layered Architecture”).

2. A written explanation, in your own words, of how your choice of architectural style affects the system design by minimizing coupling and maximizing cohesion in the system your team has implemented.
3. A cover page showing the title (deliverable name), date, team number, and team member names and student numbers.

Requirements

- The UML system decomposition diagram must represent the features your team implements in Part 2 and must match your submitted code.
- The system decomposition must be based on an architectural style covered in class (for example, N-tier, MVC, repository, etc.).
- Your diagram must show all entity, control, and boundary classes for the Part 2 features. Show only class names and do not show attributes or operations.
- Show all associations between individual classes, both within and across subsystems. Associations shown only between subsystems are not sufficient.
- Group classes into subsystems, shown as labelled UML packages.
- Group classes to minimize coupling and maximize cohesion.
- Your **document** must explain, in your own words, how your deliverable team's design minimizes coupling and maximizes cohesion.
- Use the **MyTrip** subsystem decomposition (Class Notes Section 3.3.3, pages 19 and 21) as a **reference** for formatting and organizing subsystems only.

Part 2 – DragonEndPOS Vertical Prototype (80 marks)

Default System Data

At startup, your program must load:

- **Café Menu** must include all of the following:
  - At least 10 menu items, including at least 3 sandwiches, 4 hot beverages, and 3 cold beverages.
  - Each menu item must have a name, category, and price.
  - Each menu item must reference the ingredients required to prepare it and the quantity of each.

- 11 unique users, including all the following:
  - 5 unique customers
    - Each must have a name, unique username, unique loyalty points number, email address, home address, loyalty points balance, and account credit balance.
    - Each customer must have 5 orders in their order history.

    - 2 unique kitchen staff, with name, unique username, and unique staff ID.
    - 2 unique cashiers, with name, unique username, and unique staff ID.
    - 1 manager, with name, unique username, and unique staff ID.
    - 1 system administrator, with name, unique username, and unique staff ID.

- **Order History** for each customer must include all of the following:
    - 5 orders with a status of Picked Up or Cancelled.
    - Each order must include an order number, items purchased, final price including tax, amount charged, loyalty points used, account credit used, loyalty points earned, order status, and type of purchase (online or in person).
    - Across all customers, the history must include at least one cancelled order that used loyalty points and account credit to pay for the order, at least one order that used loyalty points to pay for the order, at least one order that used account credit to pay for the order, and at least one order that earned loyalty points.
    - Customers' loyalty point and account credit balances must be consistent with the order history.

All data is held in memory only (no database in this deliverable). Store the data objects in memory. You will be deducted marks for not storing data as objects.

When the program restarts:

- The kitchen order queue will be reset to empty.
- All orders placed during the previous program execution will be removed.
- All customer order histories, loyalty points balances, and account credit balances will be reset to their default values.

Functional Requirements

Refer to the DragonEndPOS Project Overview for the details of each feature. The requirements below list only what is specific to this deliverable (additions, modifications, or simplifications for each feature). Where they differ from the Project Overview, the requirements in this document take precedence. Post any clarifying questions on the course forum.

Your team is responsible for identifying all requirements in the Project Overview that apply to each feature. Marks will be deducted for each missing item detailed for a feature, in this document or in the Project Overview.

1. **User Identification (Not True Authentication)**
    - Users enter their name or username on the startup form; the system verifies the name against the in-memory data structure and displays the appropriate interface based on their user type (Customer, Kitchen Staff, Cashier, Manager, or System Administrator).
    - Customer and Kitchen Staff can use the features of the program after their name or username is verified.

- The Cashier, Manager, and System Administrator interfaces should display only the user type and name after verifying the username/name, with no other features.
- Exiting the system returns the user to the startup form and clears the previous user's data to prevent information from carrying over between users.

Customer Features (Must Implement)

2. **View Café Menu**
   - See Project Overview, Section 2: Customer, Online Order.
   - Display menu items grouped by category.
3. **Place an Online Order**
   - See Project Overview, Section 2: Online Order, Ordering Application; Section 3: Business Rules.
   - Display the subtotal, tax, final price including tax, loyalty points redeemed, account credit applied, and the amount to be paid before the order is submitted. Tax is calculated on the full subtotal before loyalty points or account credit are applied.
4. **View Customer Online Dashboard**
   - See Project Overview, Section 2: Online Dashboard; Section 3: Business Rules.
   - Display the current order (In Progress or Completed).
   - Orders placed in this prototype cannot reach Picked Up or Cancelled, so they remain as a current order and do not update the order history.

Kitchen Staff Features (Must Implement)

5. **View Order Queue**
   - See Project Overview, Section 2: Staff: Kitchen, Order Queue Process.
   - For each order in the queue, display the order number, customer name, and order date and time.
6. **View Order Details**
   - See Project Overview, Section 2: Staff: Kitchen.
7. **Complete an Order**
   - See Project Overview, Section 2: Staff: Kitchen, Order Queue Process; Section 3: Business Rules.

Business Rules

All business rules in the Project Overview apply, except where excluded below. The following rules are specific to this deliverable:

- Orders placed in this prototype are recorded as paid, with the amount charged equal to the amount to be paid after loyalty points and account credit are applied. No payment system is called.
- Only the In Progress to Completed transition is used in this prototype.

- A customer may have only one current order at a time. If the customer already has a current order, the system must prevent placing a new order and display a message stating “Only one order can be placed at a time”. Because orders cannot reach Picked Up or Cancelled in this prototype, each customer can place only one order per program run.

Excluded from this Prototype:

- No payment system calls, and orders are recorded as paid.
- No order cancellation
- No in-person orders or cashier features (no Picked Up transition)
- No inventory tracking, ingredient deduction, or sold-out items
- No manager or administrator features
- No reports, audit trail, or session timeout
- No database or file persistence
- No AODA accessibility compliance (Project Overview, Section 4.1)

Grading Breakdown

Part 1- System Decomposition Diagram 20 marks
Part 2 - Functional Implementation 80 marks

Implementation Requirements

The following requirements specify the technical environment and design standards for DragonEndPOS.

- The course virtual machine (VM) or OpenStack, running Linux Ubuntu and Qt Creator 6.0.2, is what you will use to implement Part 2, and the TAs will use to evaluate your code. Do not run apt upgrade, update Qt Creator from within the IDE, or install a newer Qt version alongside it.
- All source code must be written in C++, designed and implemented at the level of a 3rd-year undergraduate Computer Science student, following standard UNIX programming conventions, and using advanced OO programming techniques such as polymorphism and design patterns.
- Only libraries already provided as part of the course VM may be used.
- DragonEndPOS must provide a graphical user interface built with Qt widgets. User features should be easily navigable as menu items, dialog boxes, or pop-up menus. The look and feel should be professional and consistent with commercially available UIs.

- The interface must clearly distinguish between user roles and keep navigation patterns, terminology, and placement of common controls consistent between roles.
- The DragonEndPOS system will run on a single host.

Submission Requirements

Submit your code as a single zip file named team_#_D1.zip (# is your team number in Brightspace). The submission must be fully turnkey: the TA must be able to extract the zip file, open the .pro file in Qt Creator, and build and run your application on the course VM in under three (3) minutes, without changing any file paths, settings, or configuration options. No manual installation of libraries, environment setup, or directory relocation is permitted.

The zip file must contain:

- All source code, resources, and data files
- The Qt project file (.pro), created with the qmake build system (do not use CMake), and of the all .cpp, .h, and .ui files
- A README file with:
    - Installation, build, and launch instructions
    - User account names or usernames for testing the application (all 11 default users)
    - A list of any features or requirements that were not implemented or are only partly implemented
- Your Part 1 document as a PDF named team_#_D1.pdf

The zip file must not contain a build folder, object files, or compiled executables. The project must not use hardcoded absolute paths in the .pro file or anywhere else.

Part 2 is graded only if the project compiles and executes. If the project does not open, build, or execute in Qt Creator on the course VM as described above, Part 2 will receive 0 marks. Every team member is responsible for verifying that the submitted zip file builds and executes before the deadline**.**

Format Requirements

- Include your Part 1 document in your zip file as a PDF named team_#_D1.pdf. Do not submit it separately.
- The document must be typed, legible, and professional, with a cover page showing the title, date, team number, and team member names and student numbers.

- All UML diagrams must be created using professional drawing tools. Hand-drawn diagrams will not be accepted.
- The UML system decomposition diagram must be embedded in the PDF. If a link to a diagram is given instead, Part 1 will receive 0 marks.
- Documents that do not conform to these specifications will earn a grade of zero.